

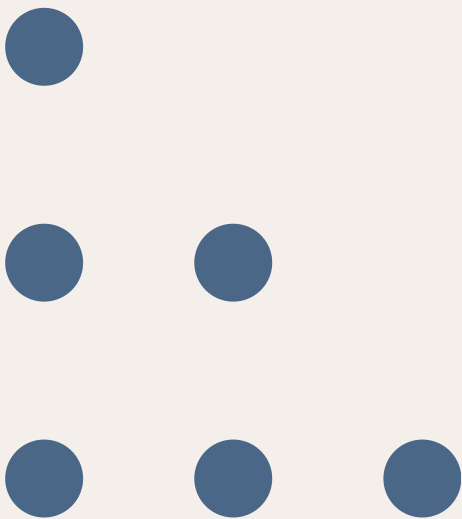


RETAIL DW

USIU-AFRICA

DSA 2040

SS2026





Our Team

Mark Kamau
Enzo Biket

Introduction

- Retail transactional data was scattered across three isolated operational silos, preventing unified reporting.
- Disparate Ingestion Streams:
 1. In-Store Sales (CSV): Text-formatted dates and duplicate transactional rows.
 2. Online Platform (JSON): Semi-structured logs with nested order timestamps and customer emails.
 3. Partner Sales (Excel): Spreadsheet records missing explicit quantities and unit prices.
- Core Objective: Build a centralized PostgreSQL data warehouse (retail_dw) to analyze revenue trends, product performance, customer behavior, and regional sales channels.

The Dimensional Model

Design Choice

Kimball-style Star Schema optimized for high-speed read operations and simplified query aggregation over Snowflake structures.

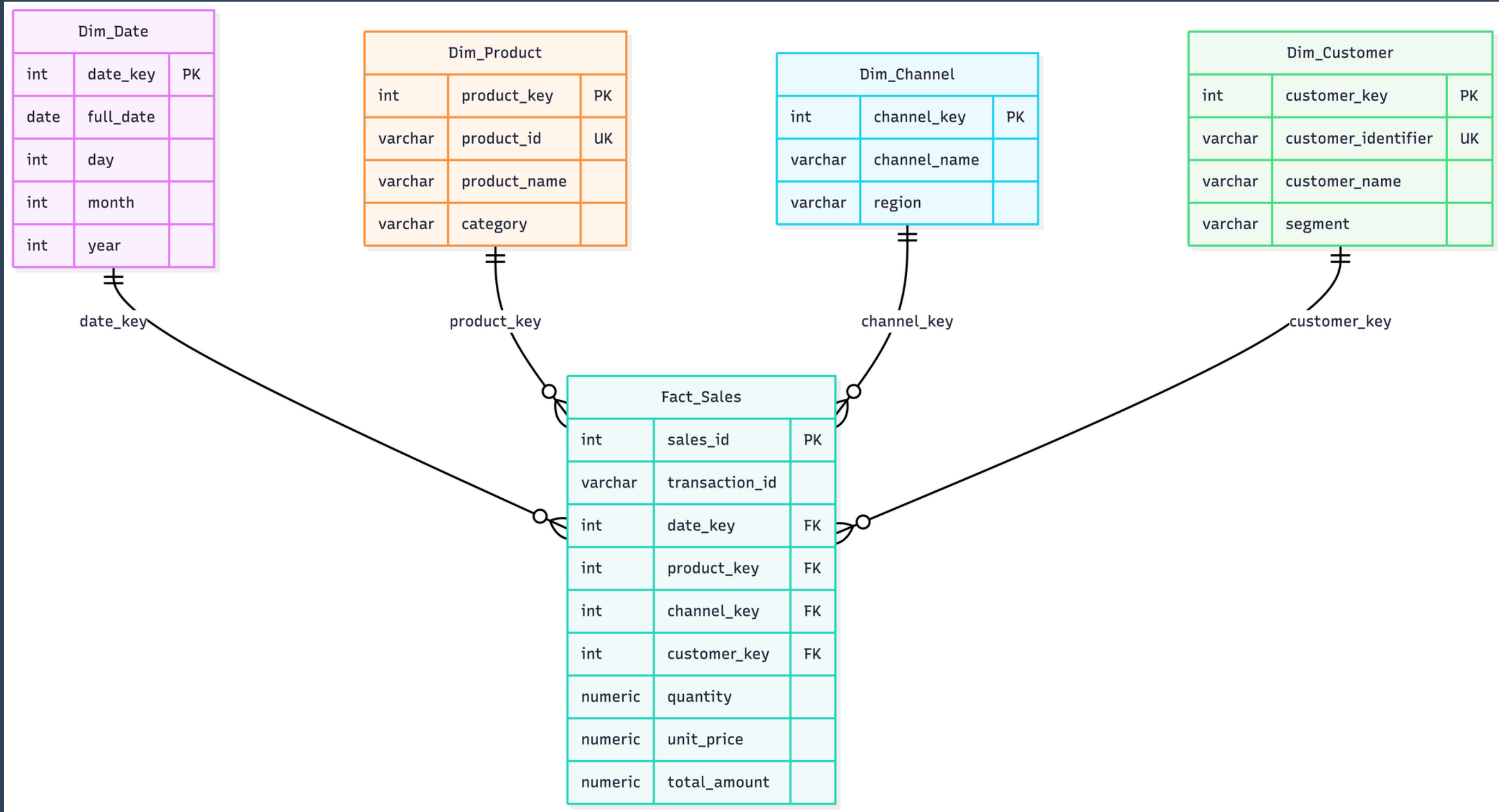
Key Dimensions Built

- Dim_Date (Time-series trends via standard YYYYMMDD keys)
- Dim_Product (Categorical inventory catalog mapping)
- Dim_Channel (In-Store, Online, and Partner geographic regional tracking)
- Dim_Customer (Behavioral tracking identifier with default fallback options)

Central Fact Table

Fact_Sales containing system-generated surrogate keys and core quantitative measures.

Model



Ingestion and Staging Strategy

Staging-Table Pattern

Decouples raw ingestion landing areas from clean, production relational storage layers.

Python Preprocessing

- Parsed raw JSON objects to unpack nested text strings and ISO timestamps.
- Cleaned and standardized text-based store dates into a uniform layout.
- Handled missing partner metrics mathematically on the fly:

Database optimization strategies

Table Partitioning

Implemented Range Partitioning on our fact table based on date keys

- Creates individual physical tables (Fact_Sales_Y2003, Fact_Sales_Y2004, Fact_Sales_Y2025).
- Bypasses scanning irrelevant years completely during execution (Partition Pruning).

Strategic Indexing

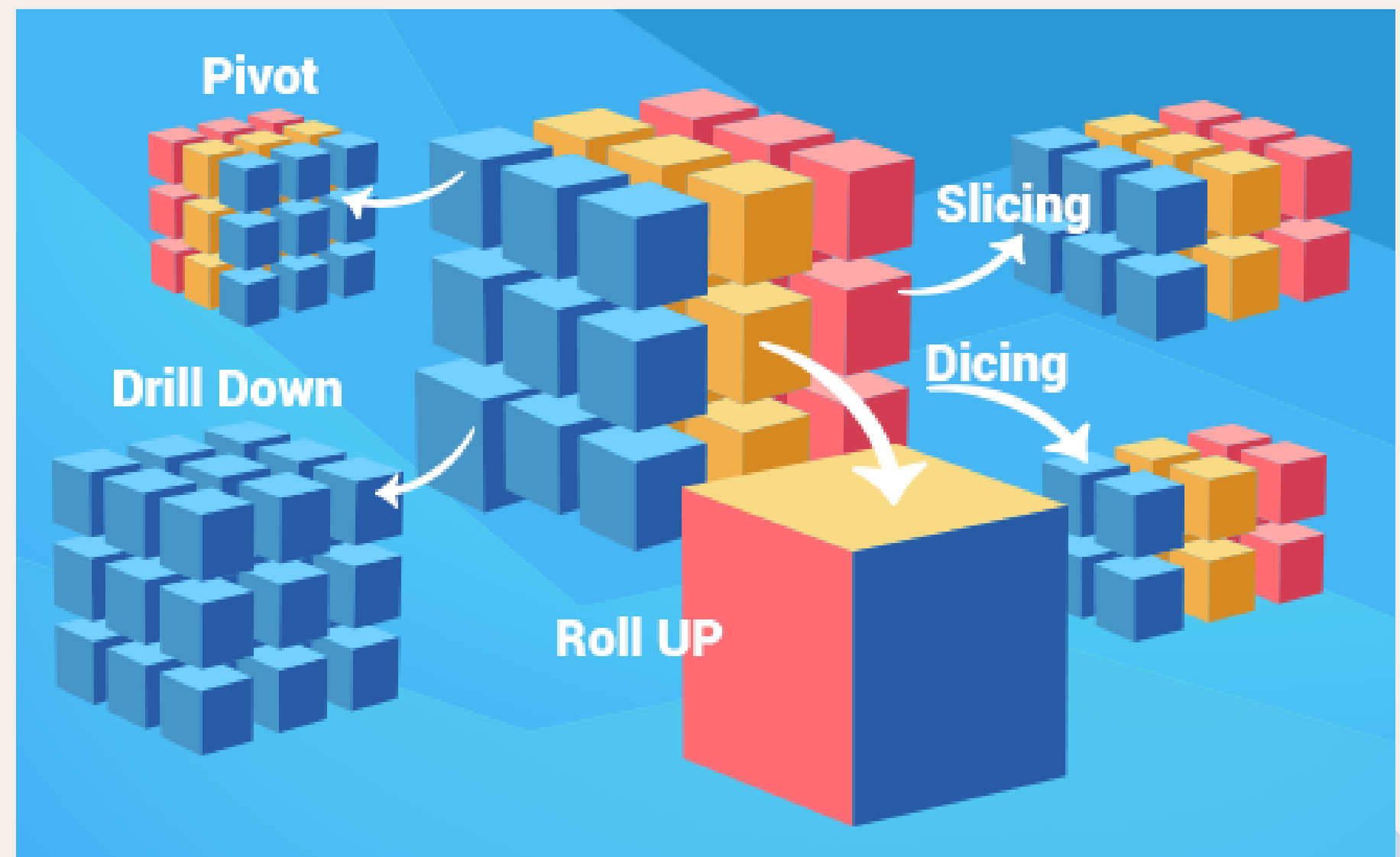
Built custom B-Tree indexes over heavily joined dimension foreign keys.

Materialized Views

Pre-calculates and caches complex, slow analytical operations directly onto the physical disk (mv_monthly_sales_summary).

Multi-dimensional OLAP Analysis

- Roll-Up: Ascended the time hierarchy to show category performance nested up to yearly subtotals and grand totals.
- Drill-Down: Descended from macro-level categories down to granular product names and units sold.
- Slice: Narrowed the data cube down to isolate performance metrics exclusively for the Online storefront.
- Dice: Intersected three dimensions simultaneously (In-Store AND Nairobi CBD AND 2003).



Performance Evaluation

The Benchmark Proof

Used query plans to trace timing before and after optimization.

Evaluation Setup	In-Engine Query Plan Operations	Planning Time (ms)	Execution Time (ms)	Performance Gain (%)
Unoptimized Base Table (Fact_Sales)	Seq Scan on Fact_Sales (Scans	0.461	2.012	Baseline
Optimized & Partitioned Table	Seq Scan on Fact_Sales_Y2003	0.998	1.004	100.4 % Faster
Materialized View Summary Cache	Index Scan on cached	0.112	0.069	2815.9 % Faster



Conclusions and Production Recommendations

Conclusions: Successfully integrated heterogeneous data formats into an optimized Star Schema without any loss of transactional fidelity.

Recommendations for Future Scaling:

- Pipeline Automation: Orchestrate the Python pandas staging flow using scheduling engines like Apache Airflow or dbt.
 - SCD Type 2 Implementation: Introduce slowly changing dimensions on Dim_Customer to track geographic shifts over time without losing history.
 - Automated Refreshes: Automate concurrent materialized view cache updates during off-peak commercial hours.
- 



Q&A Session?



THANK
YOU!